

Ketentuan Tugas Pendahuluan

1. Tugas Pendahuluan dikerjakan secara **Individu**.
2. TP ini bersifat **WAJIB**, tidak mengerjakan = **PENGURANGAN POIN JURNAL / TES ASESMEN**.
3. Hanya **MENGUMPULKAN** tetapi **TIDAK MENGERJAKAN** = **PENGURANGAN POIN JURNAL / TES ASESMEN**.
4. Deadline pengumpulan TP Modul 7 adalah Senin, 28 Oktober 2024 pukul 06.00 WIB.
5. **TIDAK ADA TOLERANSI KETERLAMBATAN, TERLAMBAT ATAU TIDAK MENGUMPULKAN TP MAKA DIANGGAP TIDAK MENGERJAKAN**.
6. **DILARANG PLAGIAT (PLAGIAT = E)**.
7. Kerjakan TP dengan jelas agar dapat dimengerti.
8. File diupload di LMS menggunakan format **PDF** dengan ketentuan: **TP_MOD_[XX]_NIM_NAMA.pdf**
9. **SOAL TEORI WAJIB DIKERJAKAN TULIS TANGAN, TIDAK BOLEH DIKETIK!**

Contoh:

```
int searchNode_130190XXXX (List L, int X);
```

CP (WA):

- Raihan (089638482851)
- Kayyisa (085105303555)
- Abiya (082127180662)
- Rio (081210978384)

SELAMAT MENGERJAKAN^^

**LAPORAN PRAKTIKUM
MODUL 9
“ TREE ”**



**Disusun Oleh:
Shilfi Habibah - 2311104002
S1SE07-01**

**Dosen :
Yudha Islami Sulistya**

**PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
INSTITUT TEKNOLOGI TELKOM PURWOKERTO
2024**

⇒ GUIDED

Code :

```
#include <iostream>
using namespace std;

/// PROGRAM BINARY TREE

// Deklarasi Pohon
struct Pohon {
    char data;
    Pohon *left, *right, *parent;
};

Pohon *root, *baru;

// Inisialisasi
void init(){
    root = NULL;
}

// Cek Node
bool isEmpty(){
    return root == NULL;
}

// Buat Node Baru
void buatNode(char data){
    if (isEmpty())
    {
        root = new Pohon(data, NULL, NULL, NULL);
        cout << "\nNode " << data << " berhasil dibuat menjadi root." << endl;
    }
    else
    {
        cout << "\nPohon sudah dibuat." << endl;
    }
}

// Tambah Kiri
Pohon *insertLeft(char data, Pohon *node){
    if (isEmpty())
    {
        cout << "\nBuat tree terlebih dahulu!" << endl;
        return NULL;
    }
    if (node->left != NULL)
    {
        cout << "\nNode " << node->data << " sudah ada child kiri!" << endl;
        return NULL;
    }
    baru = new Pohon(data, NULL, NULL, node);
    node->left = baru;
    cout << "\nNode " << data << " berhasil ditambahkan ke child kiri " << node->data << endl;
    return baru;
}

// Tambah Kanan
Pohon *insertRight(char data, Pohon *node){
    if (isEmpty())
    {
        cout << "\nBuat tree terlebih dahulu!" << endl;
        return NULL;
    }
    if (node->right != NULL)
    {
        cout << "\nNode " << node->data << " sudah ada child kanan!" << endl;
        return NULL;
    }
    baru = new Pohon(data, NULL, NULL, node);
    node->right = baru;
    cout << "\nNode " << data << " berhasil ditambahkan ke child kanan " << node->data << endl;
    return baru;
}
```

```
// Ubah Data Tree
void update(char data, Pohon *node){
    if (isEmpty())
    {
        cout << "\nBuat tree terlebih dahulu!" << endl;
        return;
    }
    if (!node)
    {
        cout << "\nNode yang ingin diganti tidak ada!" << endl;
        return;
    }
    char temp = node->data;
    node->data = data;
    cout << "\nNode " << temp << " berhasil diubah menjadi " << data << endl;
}

// Lihat Isi Data Tree
void retrieve(Pohon *node){
    if (isEmpty())
    {
        cout << "\nBuat tree terlebih dahulu!" << endl;
        return;
    }
    if (!node)
    {
        cout << "\nNode yang ditunjuk tidak ada!" << endl;
        return;
    }
    cout << "\nData node: " << node->data << endl;
}

// Cari Data Tree
void find(Pohon *node){
    if (isEmpty())
    {
        cout << "\nBuat tree terlebih dahulu!" << endl;
        return;
    }
    if (!node)
    {
        cout << "\nNode yang ditunjuk tidak ada!" << endl;
        return;
    }
    cout << "\nData Node: " << node->data << endl;
    cout << "Root: " << root->data << endl;
    cout << "Parent: " << (node->parent ? node->parent->data : '(tidak punya parent)') << endl;

    if (node->parent)
    {
        if (node->parent->left == node && node->parent->right)
            cout << "Sibling: " << node->parent->right->data << endl;
        else if (node->parent->right == node && node->parent->left)
            cout << "Sibling: " << node->parent->left->data << endl;
        else
            cout << "Sibling: (tidak ada sibling)" << endl;
    }
}
```

```
// Fungsi main
int main() {
    init();
    buatNode('A');
    Pohon *nodeB = insertLeft('B', root);
    Pohon *nodeC = insertRight('C', root);
    insertLeft('D', nodeB);
    insertRight('E', nodeB);
    insertLeft('F', nodeC);
    insertRight('G', nodeC);

    cout << "\n== Pemanggilan Retrieve";
    retrieve(root);
    retrieve(nodeB);
    retrieve(nodeC);

    cout << "\n== Pemanggilan Find (Node B)";
    find(nodeB);

    cout << "\n== Pemanggilan Find (Node C)";
    find(nodeC);

    cout << "\n== Pemanggilan Update (Mengubah Node B menjadi Z)";
    update('Z', nodeB);

    cout << "\n== Pemanggilan Retrieve setelah Update";
    retrieve(nodeB);

    return 0;
}
```

- Deklarasi struct Pohon : terdapat data untuk menyimpan pada node, left & right yaitu pointer ke anak kiri dan kanan dan parent yaitu pointer ke node induk.
- Inisialisasi Tree : mengatur root (akar pohon) menjadi NULL atau pohon kosong
- Cek Tree kosong : mengecek apakah tree kosong dengan memeriksa apakah root == NULL
- Membuat Root : membuat node pertama sebagai root, jika sudah akan menampilkan pesan bahwa pohon sudah dibuat
- Menambahkan Child kiri dan kanan : terdapat kode insertRight/Left yaitu menambahkan node sebagai anak kiri dari node tertentu, memeriksa apakah node yang akan ditambah sudah memiliki anak kiri/kanan sebelum ditambahkan.
- Mengubah Data Node dari sebuah node menjadi data baru (void update)
- Melihat data node dari yang ditunjuk (void retrieve)
- Mencari node dengan menampilkan informasi lengkap dari node yang dimaksud, termasuk: Data node, Root tree, Parent node dan Bibling(jika ada)
- Pada bagian main yaitu membuat root A , menambahkan child kiri (B) dan kanan (C) untuk root, menambahkan child untuk B(anak kiri D dan anak kanan E), menambahkan child untuk c (anak kiri F dan anak kanan G) , menampilkan informasi node menggunakan fungsi retrieve dan find serta mengubah data node B menjadi Z

Output:

```
Node G berhasil ditambahkan ke child kanan C

== Pemanggilan Retrieve
Data node: A

Data node: B

Data node: C

== Pemanggilan Find (Node B)
Data Node: B
Root: A
Parent: 65
Sibling: C

== Pemanggilan Find (Node C)
Data Node: C
Root: A
Parent: 65
Sibling: B

== Pemanggilan Update (Mengubah Node B menjadi Z)
Node B berhasil diubah menjadi Z

== Pemanggilan Retrieve setelah Update
Data node: Z
```

-Program ini mendemonstrasikan implementasi dasar binary tree, termasuk fungsi untuk menambah, melihat, mengubah, dan mencari node dalam tree.

⇒ UNGUIDED

Code :

```
#include <iostream>
using namespace std;

// Deklarasi Pohon
struct Pohon {
    char data;
    Pohon *left, *right, *parent;
};

// Variabel global
Pohon *root, *baru;

// Inisialisasi
void init() {
    root = NULL;
}

// Cek Node
bool isEmpty() {
    return root == NULL;
}

// Buat Node Baru
void buatNode(char data) {
    if (isEmpty()) {
        root = new Pohon{data, NULL, NULL, NULL};
        cout << "Node " << data << " berhasil dibuat menjadi root." << endl;
    } else {
        cout << "Pohon sudah dibuat." << endl;
    }
}

// Tambah Kiri
Pohon *insertLeft(char data, Pohon *node) {
    if (!node || node->left != NULL) {
        cout << "Child kiri sudah ada atau node tidak valid!" << endl;
        return NULL;
    }
    baru = new Pohon{data, NULL, NULL, node};
    node->left = baru;
    cout << "Node " << data << " berhasil ditambahkan ke child kiri " << node->data << endl;
    return baru;
}

// Tambah Kanan
Pohon *insertRight(char data, Pohon *node) {
    if (!node || node->right != NULL) {
        cout << "Child kanan sudah ada atau node tidak valid!" << endl;
        return NULL;
    }
    baru = new Pohon{data, NULL, NULL, node};
    node->right = baru;
    cout << "Node " << data << " berhasil ditambahkan ke child kanan " << node->data << endl;
    return baru;
}

// Tampilkan Child dan Descendant
void tampilkanChild(Pohon *node) {
    if (!node) {
        cout << "Node tidak valid!" << endl;
        return;
    }
    cout << "Child dari node " << node->data << ": ";
    if (node->left) cout << "Kiri: " << node->left->data << " ";
    if (node->right) cout << "Kanan: " << node->right->data << " ";
    if (!node->left && !node->right) cout << "(tidak ada child)";
    cout << endl;
}
```

```

void tampilkanDescendant(Pohon *node) {
    if (!node) return;
    if (node->left) {
        cout << node->left->data << " ";
        tampilkanDescendant(node->left);
    }
    if (node->right) {
        cout << node->right->data << " ";
        tampilkanDescendant(node->right);
    }
}

// Periksa Properti BST
bool is_valid_bst(Pohon *node, char min_val, char max_val) {
    if (!node) return true;
    if (node->data <= min_val || node->data >= max_val) return false;
    return is_valid_bst(node->left, min_val, node->data) &&
        is_valid_bst(node->right, node->data, max_val);
}

// Hitung Simpul Daun
int cari_simpul_daun(Pohon *node) {
    if (!node) return 0;
    if (!node->left && !node->right) return 1;
    return cari_simpul_daun(node->left) + cari_simpul_daun(node->right);
}

// Fungsi main
int main() {
    init();
    int pilihan;
    char data, parent;
    Pohon *node;

    do {
        cout << "\n=== Menu Binary Tree ===\n";
        cout << "1. Buat Root\n";
        cout << "2. Tambah Child Kiri\n";
        cout << "3. Tambah Child Kanan\n";
        cout << "4. Tampilkan Child\n";
        cout << "5. Tampilkan Descendant\n";
        cout << "6. Periksa Properti BST\n";
        cout << "7. Hitung Simpul Daun\n";
        cout << "8. Keluar\n";
        cout << "Pilihan: ";
        cin >> pilihan;

        switch (pilihan) {
            case 1:
                if (isEmpty()) {
                    cout << "Masukkan data root: ";
                    cin >> data;
                    buatNode(data);
                } else {
                    cout << "Root sudah ada!" << endl;
                }
                break;

            case 2:
                cout << "Masukkan data parent: ";
                cin >> parent;
                cout << "Masukkan data child kiri: ";
                cin >> data;
                node = root;
                while (node && node->data != parent) {
                    node = node->left ? node->left : node->right;
                }
        }
    } while (pilihan != 8);
}

```



```
        if (node) insertRight(data, node);
        else cout << "Parent tidak ditemukan!" << endl;
        break;

    case 4:
        cout << "Masukkan data node: ";
        cin >> parent;
        node = root;
        while (node && node->data != parent) {
            node = node->left ? node->left : node->right;
        }
        if (node) tampilkanChild(node);
        else cout << "Node tidak ditemukan!" << endl;
        break;

    case 5:
        cout << "Masukkan data node: ";
        cin >> parent;
        node = root;
        while (node && node->data != parent) {
            node = node->left ? node->left : node->right;
        }
        if (node) {
            cout << "Descendant dari node " << node->data << ": ";
            tampilkanDescendant(node);
            cout << endl;
        } else {
            cout << "Node tidak ditemukan!" << endl;
        }
        break;

    case 6:
        cout << "Memeriksa apakah pohon valid BST...\n";
        cout << (is_valid_bst(root, CHAR_MIN, CHAR_MAX) ? "Valid" : "Tidak valid") << endl;
        break;

    case 6:
        cout << "Memeriksa apakah pohon valid BST...\n";
        cout << (is_valid_bst(root, CHAR_MIN, CHAR_MAX) ? "Valid" : "Tidak valid") << endl;
        break;

    case 7:
        cout << "Jumlah simpul daun: " << cari_simpul_daun(root) << endl;
        break;

    case 8:
        cout << "Keluar dari program.\n";
        break;

    default:
        cout << "Pilihan tidak valid!" << endl;
        break;
    }
} while (pilihan != 8);

return 0;
}
```

Output:

```
=== Menu Binary Tree ===
1. Buat Root
2. Tambah Child Kiri
3. Tambah Child Kanan
4. Tampilkan Child
5. Tampilkan Descendant
6. Periksa Properti BST
7. Hitung Simpul Daun
8. Keluar
Pilihan: 1
Masukkan data root: A
Node A berhasil dibuat menjadi root.

=== Menu Binary Tree ===
1. Buat Root
2. Tambah Child Kiri
3. Tambah Child Kanan
4. Tampilkan Child
5. Tampilkan Descendant
6. Periksa Properti BST
7. Hitung Simpul Daun
8. Keluar
Pilihan: 2
Masukkan data parent: B
Masukkan data child kiri: C
Parent tidak ditemukan!

=== Menu Binary Tree ===
1. Buat Root
2. Tambah Child Kiri
3. Tambah Child Kanan
4. Tampilkan Child
5. Tampilkan Descendant
6. Periksa Properti BST
7. Hitung Simpul Daun
8. Keluar
Pilihan: 4
Masukkan data node: D
Node tidak ditemukan!

=== Menu Binary Tree ===
1. Buat Root
2. Tambah Child Kiri
3. Tambah Child Kanan
4. Tampilkan Child
5. Tampilkan Descendant
6. Periksa Properti BST
7. Hitung Simpul Daun
8. Keluar
Pilihan: 5
Masukkan data node: B
Node tidak ditemukan!

=== Menu Binary Tree ===
1. Buat Root
2. Tambah Child Kiri
3. Tambah Child Kanan
4. Tampilkan Child
5. Tampilkan Descendant
6. Periksa Properti BST
7. Hitung Simpul Daun
8. Keluar
Pilihan: 6
Memeriksa apakah pohon valid BST...
Valid

=== Menu Binary Tree ===
1. Buat Root
2. Tambah Child Kiri
3. Tambah Child Kanan
4. Tampilkan Child
5. Tampilkan Descendant
6. Periksa Properti BST
7. Hitung Simpul Daun
8. Keluar
Pilihan: 7
Jumlah simpul daun: 1

=== Menu Binary Tree ===
1. Buat Root
2. Tambah Child Kiri
3. Tambah Child Kanan
4. Tampilkan Child
5. Tampilkan Descendant
6. Periksa Properti BST
7. Hitung Simpul Daun
8. Keluar
Pilihan: 8
Keluar dari program.

Process returned 0 (0x0)   execution
Press any key to continue.
```

-
- Penambahan menu untuk input data tree dari user: membuat pengguna membuat pohon secara interaktif menggunakan menu, termasuk menampilkan child dan descendant dari node tertentu.
 - Fungsi Rekursif untuk memeriksa properti Binary Search Tree (BST) : Fungsi `is_valid-bst` menggunakan batas minimum dan maksimum untuk maksimum untuk memvalidasi apakah pohon memenuhi properti BST.
 - Fungsi Rekursif untuk menghitung simpul daun : fungsi `cari-simpul-daun` menghitung jumlah node tanpa anak kiri atau anak kanan.